

L02 – Deploying an AI Model on a Simulated Edge Device

Part 1: Conceptual Setup of the Development Environment

1. Python Installation and Configuration

Python is the foundational programming language for modern AI and machine learning workflows, thanks to its extensive ecosystem and library support.

Conceptual Installation Procedure

1. Access the official Python distribution website.
2. Download the most recent stable Python 3.x release compatible with the operating system.
3. Execute the installer package.
4. Enable the configuration option “Add Python to PATH” during installation.
5. Complete installation and validation using:
`python --version`

Rationale

Adding Python to the system PATH ensures that Python executables and package management tools (pip) are globally accessible, which is essential for installing and managing machine learning frameworks.

2. TensorFlow and TensorFlow Lite Installation

TensorFlow is a full-featured machine learning framework, while TensorFlow Lite is a lightweight runtime optimized for inference on edge and embedded systems.

Conceptual Installation Steps

1. Open a terminal or command-line interface.
2. Install TensorFlow via the Python package manager:
`pip install tensorflow`
3. TensorFlow Lite is bundled within TensorFlow and becomes available automatically.

Verification

`pip show tensorflow`

Why TensorFlow Lite Is Required

Edge devices operate under constraints such as limited memory, reduced compute power, and lower energy availability. TensorFlow Lite addresses these constraints by:

- Reducing model size
- Optimizing inference execution
- Supporting quantization and hardware acceleration

3. Jupyter Notebook Installation

Jupyter Notebook provides an interactive computational environment commonly used for data science experimentation and model prototyping.

Conceptual Installation Steps

Install Jupyter using pip:

`pip install jupyter`

- 1.

Launch the notebook interface:

`jupyter notebook`

- 2.

- 3. A web-based interface opens, enabling interactive execution and visualization.

Purpose in AI Development

Jupyter facilitates iterative experimentation, documentation, and visualization—key components in professional AI workflows.

Part 2: Conceptual Creation and Training of an AI Model

1. Neural Network Architecture Design

Dataset Overview

The MNIST dataset consists of grayscale images of handwritten digits (0–9), each with a resolution of 28×28 pixels.

Conceptual Model Architecture

Layer	Type	Parameters	Purpose
Input	Flatten	28×28 - 784	Converts the image matrix into a vector
Hidden	Dense	128 neurons, ReLU	Learns nonlinear patterns
Output	Dense	10 neurons, Softmax	Produces class probabilities

Activation Function Justification

- **ReLU:** Improves convergence and avoids vanishing gradients.
- **Softmax:** Converts logits into normalized probability distributions suitable for multi-class classification.

2. Data Loading and Preprocessing

Conceptual Workflow

1. Load MNIST using a dataset loader provided by Keras.
2. Split data into training and test sets.
3. Normalize pixel intensities:
$$\text{normalized_value} = \text{pixel_value} / 255$$

Why Normalization Is Critical

- Ensures numerical stability
- Improves gradient descent efficiency
- Prevents dominance of large input values

3. Model Compilation and Training

Compilation Parameters

Component	Selection	Justification
Optimizer	Adam	Adaptive learning rate optimization
Loss Function	Sparse Categorical Crossentropy	Suitable for integer-labeled classes
Metric	Accuracy	Measures classification correctness

Training Process

- The model iteratively adjusts weights through backpropagation.
- Training occurs across multiple epochs to minimize loss.
- Validation data is used to detect overfitting.

Part 3: Conceptual Conversion and Saving of the Model

1. Model Conversion to TensorFlow Lite

Purpose of Conversion

Standard TensorFlow models are optimized for training, not deployment. TensorFlow Lite conversion:

- Strips training-only metadata
- Optimizes execution graphs
- Enables deployment on constrained devices

Conceptual Conversion Steps

1. Load the trained Keras model.
2. Initialize a TensorFlow Lite converter.
3. Convert the model into .tflite format.

2. Saving the Converted Model

The converted model is saved as a binary .tflite file, which:

- Is platform-independent

- Can be embedded into edge devices
- Supports fast inference execution

Part 4: Conceptual Deployment on a Simulated Edge Device

1. TensorFlow Lite Interpreter Setup

Conceptual Deployment Steps

1. Load the .tflite model using the TensorFlow Lite Interpreter.
2. Allocate memory for tensors.
3. Retrieve metadata for input and output tensors.
4. Prepare input data in the required shape and datatype.
5. Execute inference.

2. Inference and Result Interpretation

Conceptual Inference Process

- Input data is passed through the optimized computation graph.
- The output tensor returns probability scores for each digit class.
- The predicted label corresponds to the highest probability value.

Validation

Comparing predicted results with known labels confirms correct deployment and inference behavior.